Manuel de programmation - pili-pos v1.0.0

Ce manuel s'adresse au distributeur (nuagemagique.dev) ou à un intégrateur tiers autorisé. Il décrit la **configuration du catalogue** (catégories, articles, modificateurs, traductions, codes TVA) et la **gestion des utilisateurs**.

Attention : **Ces opérations ne sont PAS accessibles à l'opérateur du restaurant.** Elles requièrent l'accès maintenance au kiosque (combinaison de touches cachée, mot de passe `root`).

1. Accès en mode programmation

1.1 Console maintenance

Sur le kiosque : 1. Brancher un clavier USB. 2. Presser `Ctrl + Alt + F2` (séquence cachée). 3. Se connecter en `root` (mot de passe noté à l'installation).

1.2 Outils disponibles

- `sqlite3` : modification directe de la base SQLite (méthode privilégiée pour la programmation du catalogue).
- `vi` : édition des fichiers de configuration.
- `journalctl -u pili-pos.service` : consultation des logs.
- `systemctl restart pili-pos.service` : redémarrer le service après modifications majeures.

2. Gestion des langues

La base est livrée avec 5 langues actives :

Code	Nom
`fr`	Français
`nl`	Nederlands
`de`	Deutsch
`en`	English
`zh`	中文

Pour désactiver une langue (sans la supprimer) :

```sql
UPDATE languages SET active = 0 WHERE code = 'nl';
```

3. Gestion des catégories de menu

3.1 Créer une catégorie

```sql
-- 1. Créer la catégorie elle-même
INSERT INTO categories (sort_order, color, active)
VALUES (1, '#F2A9BD', 1);

-- Noter l'ID auto-incrémenté
SELECT last_insert_rowid();    -- supposons 5

-- 2. Insérer les traductions du nom (au moins fr et zh)
INSERT INTO translations (entity_type, entity_id, field, lang, value)
VALUES
    ('category', 5, 'name', 'fr', 'Sushi'),
    ('category', 5, 'name', 'zh', '□ □ '),
    ('category', 5, 'name', 'nl', 'Sushi'),
    ('category', 5, 'name', 'en', 'Sushi');
```

3.2 Modifier l'ordre d'affichage

```sql
UPDATE categories SET sort_order = 2 WHERE id = 5;
```

3.3 Désactiver une catégorie (sans supprimer)

```sql
UPDATE categories SET active = 0 WHERE id = 5;
```

Attention : Ne JAMAIS faire de `DELETE FROM categories` si la catégorie contient des `menu_items` référencés dans des `order_lines` - la FK casse.

4. Gestion des articles (`menu_items`)

4.1 Créer un article

```sql
INSERT INTO menu_items
    (category_id, price_cents, vat_code, vat_category,
     kitchen_printer, sort_order, active)
VALUES
    (5, 850, 'B', 'food', 1, 0, 1);
-- 850 centimes = 8,50 €
-- vat_code B = 12% (service salle)
-- kitchen_printer 1 = à envoyer en cuisine

SELECT last_insert_rowid();   -- supposons 42

-- Traductions du nom
INSERT INTO translations (entity_type, entity_id, field, lang, value)
VALUES
    ('menu_item', 42, 'name', 'fr', 'Saumon roll x8'),
    ('menu_item', 42, 'name', 'zh', '□ 文□ □  x8'),
    ('menu_item', 42, 'name', 'nl', 'Zalm roll x8'),
    ('menu_item', 42, 'name', 'en', 'Salmon roll x8');
```

4.2 Codes TVA (rappel)

vat_code	Taux	vat_category	Usage
A	21 %	beverage	Boissons, alcools
B	12 %	food	Cuisine consommée en salle
C	6 %	food	À emporter (cuisine)
D	0 %	zero_rated	Produits exonérés
X	-	out_of_scope	Hors champ TVA

4.3 Modifier le prix d'un article

```sql
UPDATE menu_items SET price_cents = 900 WHERE id = 42;
```

Attention : Les `order_lines` existantes conservent le prix au moment de leur ajout (champ dénormalisé `unit_price_cents`) - le changement n'affecte que les futures commandes. C'est le comportement attendu fiscalement.

4.4 Désactiver un article (rupture, retrait carte)

```sql
UPDATE menu_items SET active = 0 WHERE id = 42;
```

Il disparaît de la grille de menu pour les nouvelles commandes mais reste consultable dans l'historique fiscal.

5. Gestion des modificateurs

5.1 Créer un groupe de modificateurs

```sql
-- Groupe de modificateurs "Sauces" pour le saumon roll (item 42)
INSERT INTO modifier_groups (min_select, max_select, menu_item_id)
VALUES (0, 2, 42);

SELECT last_insert_rowid();   -- supposons 10

-- Traduction du nom du groupe
INSERT INTO translations (entity_type, entity_id, field, lang, value)
VALUES
    ('modifier_group', 10, 'name', 'fr', 'Sauces'),
    ('modifier_group', 10, 'name', 'zh', '□ □ '),
    ('modifier_group', 10, 'name', 'en', 'Sauces');
```

5.2 Ajouter des options au groupe

```sql
-- "Sauce soja" – gratuite
INSERT INTO modifiers (group_id, price_cents, sort_order)
VALUES (10, 0, 0);
SELECT last_insert_rowid();   -- supposons 100

INSERT INTO translations (entity_type, entity_id, field, lang, value)
VALUES
    ('modifier', 100, 'name', 'fr', 'Sauce soja'),
    ('modifier', 100, 'name', 'zh', '□ □ '),
    ('modifier', 100, 'name', 'en', 'Soy sauce');

-- "Wasabi" – supplément 50 centimes
INSERT INTO modifiers (group_id, price_cents, sort_order)
VALUES (10, 50, 1);
SELECT last_insert_rowid();   -- supposons 101

INSERT INTO translations (entity_type, entity_id, field, lang, value)
VALUES
    ('modifier', 101, 'name', 'fr', 'Wasabi'),
    ('modifier', 101, 'name', 'zh', '□ □ '),
    ('modifier', 101, 'name', 'en', 'Wasabi');
```

6. Gestion des utilisateurs

6.1 Créer un utilisateur

```sql
INSERT INTO users (insz, name, role, active)
VALUES ('85073003328', 'Marie Dupont', 'cashier', 1);
```

Rôles autorisés : `admin`, `cashier`, `waiter`. Voir `PROFILS_UTILISATEURS_FR.md` pour le mapping SCE 2.0.

6.2 Définir un code PIN

Le PIN doit être haché avec bcrypt (coût 12). En l'absence d'outil CLI bcrypt, le distributeur peut le faire depuis le terminal :

```
# Sur la machine du distributeur
htpasswd -bnBC 12 "" "1234" | tr -d ':\n'
# Sortie : $2y$12$abcdef...
```

Puis :

```
UPDATE users
SET pin_hash = '$2y$12$abcdef...'
WHERE insz = '85073003328';
```

L'utilisateur peut maintenant se connecter avec son INSZ + PIN 1234.

6.3 Désactiver un utilisateur

```
UPDATE users SET active = 0 WHERE insz = '85073003328';
```

L'utilisateur ne peut plus se connecter. Les événements fiscaux qu'il a générés restent intacts (l'INSZ est conservé comme identifiant historique).

Attention : Ne JAMAIS faire `DELETE FROM users` pour un utilisateur ayant des événements fiscaux à son nom - la FK depuis `fiscal_events` empêcherait la suppression, et même si elle réussissait par cascade (non configurée ici), cela violerait l'obligation de conservation.

6.4 Réinitialiser un PIN oublié

```
UPDATE users SET pin_hash = NULL WHERE insz = '85073003328';
```

Un Serveur ou un Apprenti peut alors se connecter sans PIN, puis en fixe un nouveau en touchant son nom sur l'écran de commande. Un Gérant choisit un nouveau PIN dès sa connexion suivante ("Choisissez votre PIN") : il ne peut pas se connecter sans.

7. Configuration de la période de réservation

La `booking_period_id` est une UUID régénérée automatiquement à chaque rapport Z. Pour la **réinitialiser manuellement** (cas exceptionnel - uniquement à l'installation initiale) :

Cette UUID vit dans `AppState.booking_period_id` en mémoire et est régénérée au prochain Z. Pas de modification SQL nécessaire en exploitation normale.

8. Désactiver / activer le mode formation

Le mode formation est un drapeau dans `pos.json` :

```
vi /etc/pili-pos/pos.json
```

Modifier `"training_mode": false` correspond à `true`. Puis :

```
systemctl restart pili-pos.service
```

Attention : Ne JAMAIS laisser `training_mode: true` à la livraison au client - toutes les ventes deviendraient des événements T (formation) et NE compteraient PAS fiscalement.

9. Modifier les informations du restaurant

Toute modification de `restaurant_name`, `restaurant_address`, `restaurant_vat`, `restaurant_est_no` se fait dans `pos.json` et nécessite un redémarrage du service.

Attention : Modifier la TVA ou le `pos_id` après mise en production implique une déclaration au SPF Finances et potentiellement une re-certification.

10. Sauvegarde et restauration

10.1 Sauvegarde manuelle

```
sqlite3 /var/lib/pili-pos/pos.db ".backup /tmp/pos-snapshot.db"
```

10.2 Restauration

Attention : Opération **DESTRUCTIVE** - la base courante est remplacée. À effectuer uniquement après arrêt du service et en cas de nécessité (panne disque, corruption avérée).

```
systemctl stop pili-pos.service
cp /tmp/pos-snapshot.db /var/lib/pili-pos/pos.db
chown pili-pos:pili-pos /var/lib/pili-pos/pos.db
chmod 0640 /var/lib/pili-pos/pos.db
systemctl start pili-pos.service
```

Toute restauration doit être déclarée : 1. Au commerçant (qui devra vérifier la cohérence de ses sauvegardes papier). 2. Le cas échéant au SPF Finances si la perte couvre une période où des ventes ont été effectuées et non remontées.

11. Migrations de schéma

Le schéma SQLite évolue via le runner versionné (`crates/pos-db/src/migrate.rs`). Les migrations sont automatiques au démarrage : si une nouvelle version est nécessaire, elle est appliquée dans une transaction `BEGIN IMMEDIATE` et tracée dans la table `_migrations_applied`.

Pour lister les migrations appliquées :

```
SELECT version, name, applied_at
FROM _migrations_applied
ORDER BY version;
```

Toute nouvelle migration introduite par un patch pili-pos suivra le même mécanisme - aucune intervention manuelle n'est requise du distributeur sur site.

12. Diagnostic et logs

12.1 Logs du service

```
# En direct
journalctl -fu pili-pos.service

# Dernière heure
journalctl -u pili-pos.service --since '1 hour ago'

# Logs fichier (rotation quotidienne)
ls /var/lib/pili-pos/logs/
tail -f /var/lib/pili-pos/logs/pos.log.YYYY-MM-DD
```

12.2 Diagnostic FDM

```
# Test brut de la connexion HTTP
curl -X POST http://192.168.1.20:8080/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "query { Status(language: FR) { device { fdmId fdmDateTime } } }"}'
```

Réponse attendue : JSON contenant l'identité du FDM. Si pas de réponse : problème réseau ou FDM hors-ligne.

12.3 Diagnostic imprimante

```
# Si TCP
nc -zv 192.168.1.10 9100
```

```
# Test d'impression caractères ESC/POS minimal
echo -e "Test impression\n\n\n\x1bm" | nc -w 1 192.168.1.10 9100
```

13. Procédure de release par le distributeur

À partir du dépôt git, chaque release suit la séquence :

```
# 1. S'assurer que tout est commité, tests verts
cargo test --workspace
cargo clippy --workspace -- -D warnings
cargo fmt --all --check

# 2. Build de certification de référence (conteneur épinglé), qui
#    vérifie aussi le périmètre du binaire (commandes en ligne
#    présentes, facturation B2B absente) et
#    produit le rapport build_info_*.txt (SHA-1, SHA-256, MD5)
./deploy/build-release.sh --docker

# 3. Reproductibilité : deux builds depuis un arbre nettoyé, même SHA-1
#    (lancé aussi par la CI, .github/workflows/image.yml, tâche repro)
scripts/check_repro.sh

# 4. Le SHA-1 à déclarer
grep '^SHA-1' deploy/dist/build_info_*.txt

# 5. Tag git signé GPG
git tag -s vX.Y.Z -m "pili-pos vX.Y.Z - <résumé>"
git push origin vX.Y.Z

# 6. Construction de l'image kiosque (mkosi), profils production et
#    admission ; le SHA-1 du binaire embarqué est vérifié dans l'image
sudo deploy/image/build.sh production
sudo deploy/image/build.sh admission

# 7. Déclaration au SPF Finances si modifications fonctionnelles
# Voir Manuel_Intro_FR §C - procédure post-cert (délai 15 jours)
```

14. Anti-pattern à éviter

NON NE PAS FAIRE	✓ FAIRE À LA PLACE
`DELETE FROM fiscal_events`	Aucune suppression - exigence légale
`DELETE FROM users WHERE insz = ...`	`UPDATE users SET active = 0 ...`
`UPDATE fiscal_events SET ...`	Aucune modification - émettre une correction via événement séparé
`DELETE FROM orders WHERE status = 'closed'`	Conservation indéfinie
Bypass de FDM en production	Bloquer la vente - exigence cert
`training_mode: true` en production	Toujours `false` après installation
Suppression d'une catégorie ayant des items	`UPDATE ... SET active = 0`
Changement de `pos_id` ou `restaurant_vat` sans déclaration SPF	Procédure post-cert obligatoire